Alexandria Engineering Journal 128 (2025) 153–165

Contents lists available at ScienceDirect

Alexandria Engineering Journal

journal homepage: www.elsevier.com/locate/aej

Original article

Machine learning algorithm for detecting suspicious email messages using Natural Language Processing *NLP*

Ghadah Alkhodhairy [a], Kashif Saleem [b,*]

[a] Cybersecurity Master Program, College of Computer & Information Sciences (CCIS), King Saud University, Riyadh 12372, Saudi Arabia
[b] Center of Excellence in Information Assurance (CoEIA), King Saud University, Riyadh 12372, Saudi Arabia

ARTICLE INFO

Keywords:
Phishing
Machine learning
Natural language processing
SVM classification
Cyber threat detection

ABSTRACT

Email-based cyber threats continue to evolve, presenting challenges to traditional security measures that struggle to keep pace with sophisticated attack methods. This study introduces a machine learning-driven framework that integrates Natural Language Processing (NLP) with a Support Vector Classifier (SVC) to enhance email threat detection. The proposed model utilizes feature extraction techniques, including one-hot encoding, TF-IDF vectorization, and BERT embeddings, to capture linguistic patterns indicative of malicious emails. The model was trained and evaluated on a benchmark dataset, achieving an accuracy of 98.65%, demonstrating superior performance over conventional spam detection systems. The results show a significant reduction in false positives and false negatives, improving email security and reliability. This research contributes to the field by presenting an optimized, scalable solution that can adapt to emerging cyber threats and enhance automated email filtering mechanisms.

1. Introduction

Over the past two decades, the internet has become integral to global connectivity, seamlessly integrating into daily life for nearly 3 billion users worldwide. Beyond enabling communication, it now drives economic growth, contributing billions annually to the global economy. However, this reliance on digital infrastructure has also escalated the risks associated with cyber threats, particularly through email-based attacks [1]. The proliferation of sensitive data online underscores the critical need for robust cybersecurity measures. Cybersecurity has emerged as a paramount concern for individuals, corporations, and institutions. The contemporary digital landscape has witnessed an unprecedented surge in the complexity of cyber threats that emanate through email-based channels. The United States Department of Justice defines cybercrime as any illegal activity done primarily through computers, including those that rely on digital records for their execution. These threats encompass a variety of malicious techniques, from subtle phishing schemes to malware distribution, which pose formidable challenges to both individuals and organizations [2].

As our lives become increasingly linked with technology, cyber threats have grown in magnitude and complexity, constituting a dangerous challenge for both individuals and organizations. Traditional email security measures, although effective to some extent, have consistently struggled to keep pace with the rapid evolution of these malevolent tactics. According to a CSIS survey conducted across eight countries, around 82% of employers are grappling with a severe shortage of cybersecurity expertise [3]. The repercussions of this talent deficit are equally glaring, with 71% of these organizations confirming tangible harm caused by this scarcity [3]. Furthermore, data from CyberSeek, a National Initiative for Cybersecurity Education (NICE)-funded initiative, emphasized the gravity of the situation in the United States. Job postings reveal that the number of unfilled cybersecurity positions has increased by over 50% since 2015 [3]. The scenario intensifies on a global scale, with projections indicating that the worldwide cybersecurity workforce deficiency could soar to an astounding 1.8 million vacant roles in the upcoming few years [3].

The failure to adapt to the threat results in a hard challenge: The generation of false positives (FP) and false negatives (FN). FPs, the incorrect identification of non-harmful emails as threats, and FNs, the failure to detect genuinely harmful emails, not only destroy the reliability of email communication but also leave individuals and organizations weak to the consequences of cyberattacks. The gradually accumulating impact of these issues makes email communication unsafe and exposes users to a range of dangers, including data breaches, financial losses, and reputational damage.

The gravity of the problem cannot be ignored, as email remains an essential mode of communication in our world of business. Email security has become a matter of great importance, necessitating innovative

* Corresponding author.
E-mail addresses: 443203981@student.ksu.edu.sa (G. Alkhodhairy), ksaleem@ksu.edu.sa (K. Saleem).

https://doi.org/10.1016/j.aej.2025.04.067
Received 21 October 2024; Received in revised form 27 March 2025; Accepted 20 April 2025
Available online 26 May 2025
1110-0168/© 2025 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

solutions to address the shortcomings of traditional security measures. This has prompted various approaches in the realm of email security, with researchers and practitioners seeking to mitigate the impact of email-based cyber threats [4]. Artificial Neural Networks (ANN) were first used to go beyond the binary classification of ham and spam emails to encompass a more nuanced three-class classification [5]. However, the potential of hybrid machine learning techniques was touched upon a lot [6,7].

With the advances witnessed in the field of cybersecurity, notable modern approaches focus on combating targeted malicious emails (TMEs). These approaches tried to detect TMEs and evaluated their effectiveness by comparing them with established methods such as SpamAssassin and ClamAV [8]. However, other approaches advanced to the use of Support Vector Machine (SVM) algorithms for the detection of malicious emails, presenting promising accuracy rates [9,10]. Nonetheless, the persistent challenge of email spamming became harder calling for feature selection techniques to enhance the performance of Spam Detection Systems (SDS) performance.

Despite advancements in spam detection, existing email security models still suffer from high false positive rates, insufficient contextual understanding of phishing attempts, and limited adaptability to emerging threats.

This research addresses these challenges by integrating NLP [11] based feature extraction with an optimized SVC model, significantly improving classification accuracy and reducing misclassification rates. This paper proposes a comprehensive approach for enhancing email security with Machine Learning (ML), Natural Language Processing (NLP), and one-hot encoding techniques. By incorporating machine learning strategies, this approach aims to raise email security by facilitating the process of feature extraction and refining the classification of potential threats.

This research presents an email security framework, which integrates ML, NLP, and one-hot encoding techniques. The integrated approach will help to effectively classify email messages that contain potential security threats. The proposed algorithm aims to improve email security by improving the efficiency of feature extraction and also by refining the process of threat classification. This will advance the security of email communication channels by increasing the accuracy of threat detection mechanisms. In essence, this system seeks to empower individuals and organizations with a state-of-the-art email security solution that offers an unparalleled level of protection against emerging and sophisticated email-based cyber threats.

The rest of the paper is organized as follows. Section 2 comprehensively reviews the related literature and mechanisms that use machine learning to detect malicious traffic. The proposed design is presented in Section 3. Section 4 describes evaluation parameters. The implementation is given in Section 5. Experimental results and comparative analysis are provided in Section 5. Section 6 discusses the findings. This study's conclusion and future work are shown in Section 7.

2. Literature studies

Cybercriminals have turned their focus to high-profile targets, such as CEOs and executives, in order to maximize profits and harm. This type of phishing, also known as whaling or CEO fraud, contains targeted and highly convincing messages that mislead people into doing activities that could result in financial losses [12]. Phishing attacks came to a new stage in the early 2020s as shown in Fig. 1. Voice phishing (vishing), SMS phishing (smishing), and classic email phishing were all used in this period. This time, artificial intelligence and machine learning are being used [13]. Cybercriminals can automate and optimize their phishing tactics with the help of AI. The new approach increases the complexity of these attacks, making them more difficult to detect [14].

2.1. Natural language processing in phishing detection

The growing threat of the internet and email-based attacks necessitated the development of more advanced detection technologies. Machine learning has emerged as a strong method for analyzing massive amounts of email data in order to detect phishing attempts. First of all, they were used to extract various features from emails, including content, URLs, sender information, and headers. These features are used as input data for the detection algorithms. ML often used supervised learning — being trained on labeled datasets containing examples of both innocent and phishing emails [15]. Various classification algorithms are applied, including SVM, Random Forest, Decision Trees, Naive Bayes, and more. These algorithms evaluate email features to classify messages as phishing or non-phishing. During training, the machine learning model learns to recognize patterns and features that differentiate phishing emails from legitimate ones. It assigns weights or coefficients to features based on the training data. The trained model is tested with a separate dataset to evaluate its performance. Metrics like accuracy, precision, recall, and F1 score are used to assess the model's effectiveness. In practice, the trained model is used to score incoming emails in real time. If the score exceeds a predefined threshold, the email is flagged as suspicious and may be blocked or quarantined [16,17].

Machine learning combined with NLP started to be used to classify emails. These systems used algorithms to analyze email content, identify deceptive language, and discern patterns commonly found in phishing emails. This shift improved the accuracy of filtering, reducing FPs and negatives. NLP advanced to consider the context and semantics of email content. Systems began to assess the intent behind the text, differentiating between emails containing routine financial transactions and those that raised suspicion. Detecting context and anomalies became essential in the battle against sophisticated phishing attacks. Moreover, linguistic patterns and sentiment analysis were used to gauge the emotional tone of messages. Phishing attempts using persuasive or urgent language were flagged as potential threats [18]. In addition to analyzing email content, they began to evaluate the effectiveness of NLP combined with several other ML models and apparently, NLP works best with SVM. The following Fig. 2 shows the popularity of various ML techniques with NLP.

2.2. Related literature

Amin et al. [19] made the first attempts to combat the growing threat of malicious emails were presented, researchers studied TMEs and their increasingly insidious nature, which has been extensively documented in recent years. Unlike conventional spam or phishing attacks aimed at tricking users into divulging personal information, TMEs serve as potent enablers of computer network exploitation and the clandestine gathering of sensitive information from targeted networks. Notably, TMEs are not isolated, singular events; rather, they manifest as coordinated and persistent campaigns that span extended periods, sometimes stretching across years. This study conducted a survey of the prevailing email filtering techniques to eliminate the damage of TMEs. Additionally, the researchers introduced novel techniques for TME detection using a Random Forest classifier. To gauge the effectiveness of their approaches, they conducted rigorous comparative analyses against two established detection methods, SpamAssassin and ClamAV.

The foundation of the new email filtering techniques relies on leveraging the power of persistent threat and recipient-oriented email features, through using a random forest classifier. This approach seeks to enhance the detection of TMEs while maintaining reasonable FP rates. During extensive testing, these techniques demonstrated the ability to accurately classify 91% of TMEs, marking a stark contrast to the 16% correctly identified by the conventional SpamAssassin+ClamAV approach.

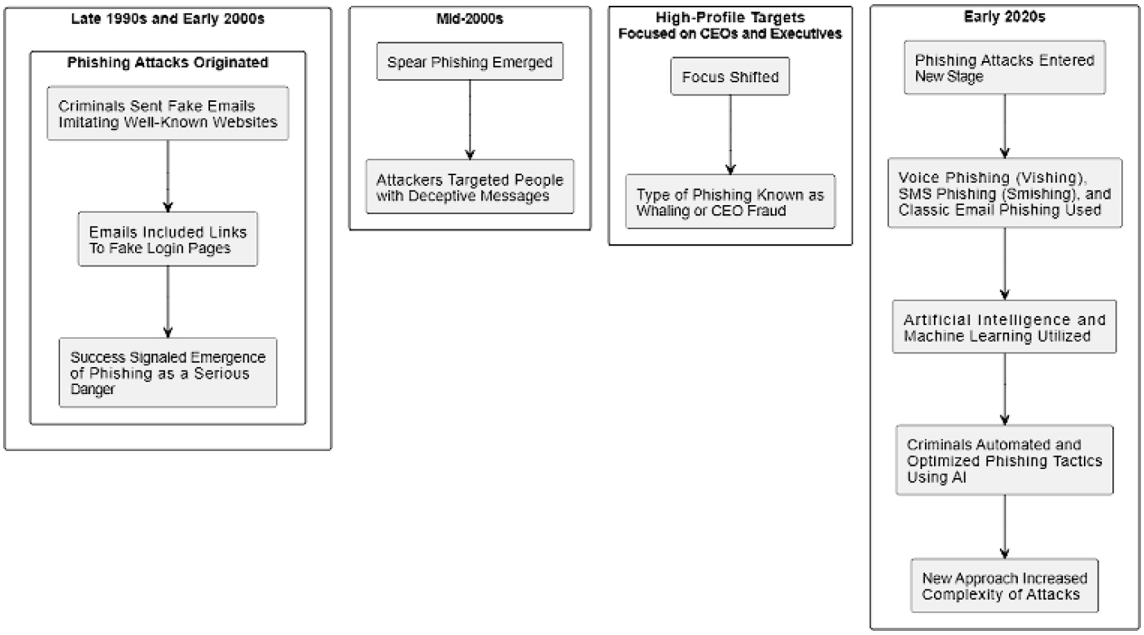


Fig. 1. Evolution of phishing attacks.

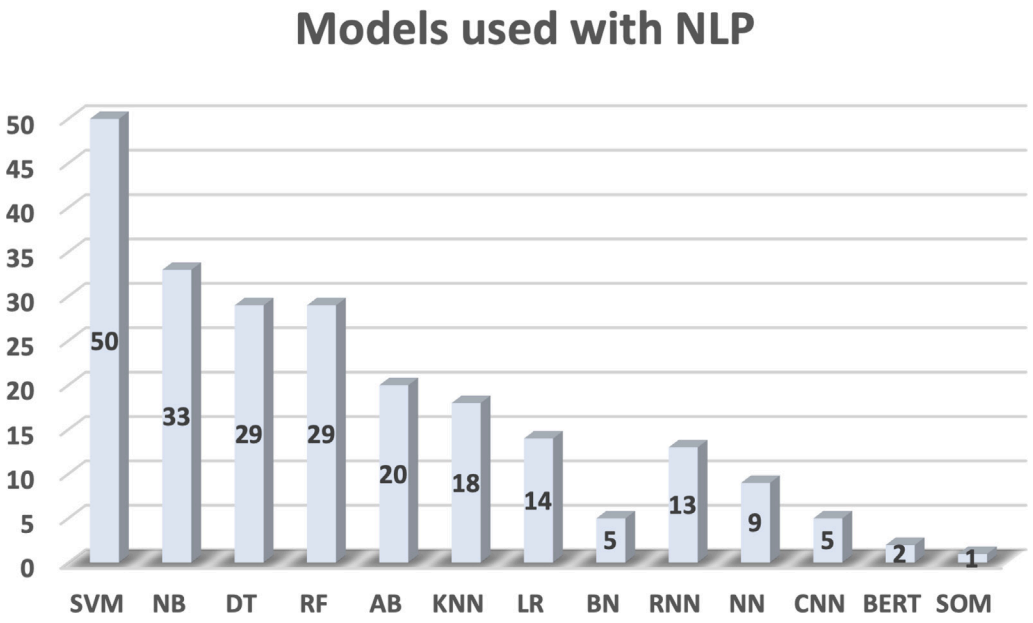


Fig. 2. The popularity of various ML techniques with NLP.

Khamis et al. [9] were the first attempts to use SVM in filtering malicious emails were made. The main point of this research was the identification of potentially valuable email header features for enhancing email spam detection. To accomplish this objective, the researchers conducted a meticulous analysis of two distinct email datasets, namely the Anomaly Detection Challenges dataset and the Cybersecurity Data Mining dataset, both sourced from reputable online repositories. The research started with Data Collection, followed by Data Pre-Processing to ensure the dataset's integrity and relevance. Feature Selection played an important role in extracting the most informative attributes from the email headers, a critical step in the spam detection process. Subsequently, the Classification phase involved the deployment of SVM algorithms, facilitated by tools such as RapidMiner Studio and Weka 3.9.2. Finally, the Detection phase aimed to evaluate the efficacy of the selected email header features in identifying spam emails. Notably, the SVM-based classification approach exhibited remarkable accuracy rates, with the CSDM2010 dataset achieving an accuracy rate of 88.80%, while the Anomaly Detection Challenges dataset demonstrated an accuracy rate of 87.20%. However, there were certain limitations in the study such as focusing on theoretical aspects and algorithmic developments, with limited exploration of practical implementation.

In 2022, Ghaleb et al. [20] focused on the persistent challenges posed by email spamming. This is especially crucial given that users face a growing challenge: a surge in legitimate communications mistakenly flagged as spam. In response to this threat, the researchers have leveraged the power of pattern recognition within email communications. This effort has led to the development of Spam Detection Systems (SDS), designed not only to identify spammers but also to effectively filter email traffic. SDS has significantly improved the capability to

detect spam by mitigating the rate of FPs and augmenting detection accuracy. However, one of the persistent challenges in the realm of spam classifiers is the sheer abundance of features. Feature Selection (FS) has emerged as a critical aspect of refining SDS performance. It plays a pivotal role in guiding feature selection algorithms to enhance the classification prowess and precision of SDS. This study introduced an innovative approach. Leveraging a wrapper technique grounded in the Multi-Objective Grasshopper Optimization Algorithm (MOGOA) for feature extraction and the recently revised Evolutionary Group Optimization Algorithm (EGOA) for training Multilayer Perceptron (MLP) models, the proposed system aims to augment the effectiveness of SDS. To evaluate the performance, the researchers conducted extensive testing using several datasets, including SpamBase, SpamAssassin, and UK-2011. The results demonstrate that the novel approach outperforms established practices in the literature, achieving remarkable accuracy improvements of up to 97.5%, 98.3%, and 96.4%, respectively.

Magdy et al. [21] used ANN based model for detecting malicious email content. The model offers a notable enhancement in performance compared to the state-of-the-art studies that have come before. The approach differs from conventional text analysis techniques, opting for content-based features, thus optimizing processing time and resource allocation. The ANN classifier demonstrates good performance, going beyond the binary classification of ham and spam emails to a more accurate three-class classification that encompasses ham, spam, and phishing emails. The tried validation of the model's performance is made on three benchmark datasets. The research team's innovative application of an ANN model configured with multiple layers, including hidden layers, resulted in an impressive maximum accuracy level on the Spambase dataset. Moreover, precision, recall, and F1 scores were also very high.

M. Dewis and T. Viana [22] developed a robust solution that leverages the potential of hybrid machine learning techniques, combining NLP and deep learning, to detect phishing and spam emails. In order to validate its efficiency, Phish Responder underwent strict experimentation. The significance of this research lies in Phish Responder's unique capacity to swiftly extract features such as URLs, email addresses, IP addresses, and phishing-related terminology from individual emails to enhance its utility in incident response scenarios. This feature enriches the incident response process and aids in expeditiously identifying and addressing threats. Results from the experiments revealed that Phish Responder achieved an average accuracy of 94% with the MLP model for numerical-based datasets. Comparative evaluations placed Phish Responder in a favorable position, demonstrating its superiority over existing approaches in the literature, with accuracy rates surpassing 97.5%, 98.3%, and 96.4% when pitted against distinct datasets.

Y. Li conducted a study [23] in which there was an experiment that meticulously compared the performance of different supervised learning methods, such as the SVM, K-Nearest Neighbor (KNN), and Naïve Bayes (NB), as well as unsupervised learning approaches, including ANN and partitional clustering. The results revealed that NB exhibited the highest accuracy, highlighting the potential suitability of NB as a model for spam filtering within the context of the employed data preprocessing techniques NLP and programming structure. NB is grounded in Bayes' Theorem, which assumes predictor independence and equal influence on results.

Our approach builds upon previous research by integrating SVC with NLP-based feature extraction and BERT to improve classification accuracy and reduce false positives. While previous studies primarily used Random Forest, Naïve Bayes, and traditional SVM models, our work demonstrates that an optimized SVC model with feature selection techniques provides higher accuracy (98.65%) and better spam filtering efficiency.

3. Methodology

Email Spam Classification Dataset available on Kaggle [24] is the benchmark dataset utilized in this study. It comprises aCSV file with details about 5172 randomly selected email files alongside their corresponding labels, facilitating the classification into spam or non-spam categories. The CSV file is a structured set of 5172 rows, each uniquely representing an individual email. There is also an array of 3002 columns — the first column anonymizes the email names and instead uses numerical designations over recipient-specific labels for privacy reasons. The final column in the dataset assigns the binary values of 1 for spam and 0 for non-spam classification for the subsequent machine learning processes. Each cell in the dataset contains the word count for the corresponding word in a specific email, the sample of this data is shown in Fig. 3.

3.1. Data collection and preprocessing

The foundational step in this study involved data acquisition, leveraging the Kaggle Email Spam Classification Dataset as the benchmark repository [24]. This dataset has a variety of email messages with diverse content for comprehensive training and testing stages. The preprocessing went through several stages. This started with lowercasing, tokenization, and then the elimination of stop words:

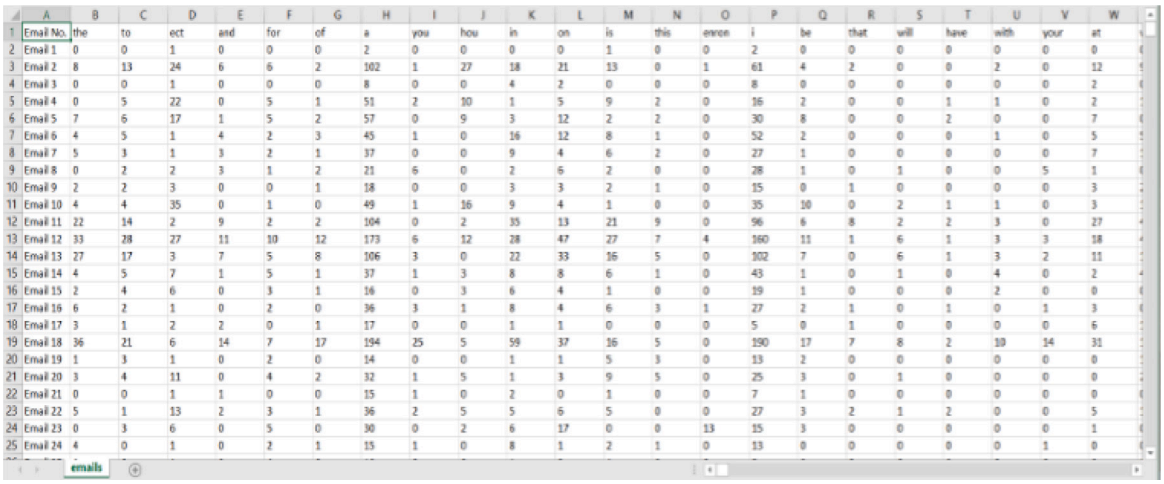
- Lowercasing: the first step in standardizing the textual content of emails. All letters within the emails were converted to lowercase. This way, any discrepancies caused by variations in letter cases were eliminated, ensuring a consistent and standard dataset.
- Tokenization: the next step was tokenization. It was done by breaking down the email content into individual units, referred to as tokens, to do a more detailed analysis of linguistic features.
- Stop Word Removal: uninformative terms such as "the", "is", and "and", were removed from the tokenized text. This helped to enhance the analyses by focusing on content-rich words, reducing noise, and refining the dataset for more meaningful feature extraction.

After these steps were performed, the dataset was ready to be fed to the NLP processing for feature extraction. .

3.2. Feature extraction

After preprocessing, relevant features were extracted from the preprocessed email content. This involved using NLP techniques, including tokenization, one-hot encoding, and TF-IDF (Term Frequency-Inverse Document Frequency) vectorization. Tokenization is primarily used on the lowercase and stop word-eliminated email content. Tokenization and Stop-Word Removal in preprocessing steps play a crucial role in enhancing model performance. As the tokenization, breaks text into meaningful units (tokens), enabling models to analyze individual words effectively. Besides, the Stop-Word Removal, eliminates common words (e.g., "the", "is", "and") that do not contribute significantly to the meaning, reducing noise and focusing on relevant content. One-hot encoding converts categorical features into a binary format to feed the model. Categorical features, such as email subject, were transformed into a binary matrix, to ease their incorporation into the feature set for subsequent model training. The next step was then to convert preprocessed text into numerical features using TF-IDF vectorization. This technique assigned weights to words based on their frequency in each email according to their occurrence across the entire dataset. This weighted representation captures the importance of words in distinguishing emails and contributes to making a feature-rich dataset. TF-IDF and one-hot encoding also involve a few limitations.

- TF-IDF: This technique quantifies the importance of words within a document relative to a corpus. Its strength lies in its simplicity and effectiveness for basic text classification [25]. However, it fails to capture the contextual relationships between words, which can limit its effectiveness in detecting sophisticated phishing emails.



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	Email No.	the	to	ect	and	for	of	a	you	hou	in	on	is	this	enron	i	be	that	will	have	with	your	at
2	Email 1	0	0	1	0	0	0	2	0	0	0	0	1	0	0	2	0	0	0	0	0	0	0
3	Email 2	8	13	24	6	6	2	102	1	27	18	21	13	0	1	61	4	2	0	0	2	0	12
4	Email 3	0	0	1	0	0	0	8	0	0	4	2	0	0	0	8	0	0	0	0	0	0	2
5	Email 4	0	5	22	0	5	1	51	2	10	1	5	9	2	0	16	2	0	0	1	1	0	2
6	Email 5	7	6	17	1	5	2	57	0	9	3	12	2	2	0	30	8	0	0	2	0	0	7
7	Email 6	4	5	1	4	2	3	45	1	0	16	12	8	1	0	52	2	0	0	0	1	0	5
8	Email 7	5	3	1	3	2	1	37	0	0	9	4	6	2	0	27	1	0	0	0	0	0	7
9	Email 8	0	2	2	3	1	2	21	6	0	2	6	2	0	0	28	1	0	1	0	0	5	1
10	Email 9	2	2	3	0	0	1	18	0	0	3	3	2	1	0	15	0	1	0	0	0	0	3
11	Email 10	4	4	35	0	1	0	49	1	16	9	4	1	0	0	35	10	0	2	1	1	0	3
12	Email 11	22	14	2	9	2	2	104	0	2	35	13	21	9	0	96	6	8	2	2	3	0	27
13	Email 12	33	28	27	11	10	12	173	6	12	28	47	27	7	4	160	11	1	6	1	3	3	18
14	Email 13	27	17	3	7	5	8	106	3	0	22	33	16	5	0	102	7	0	6	1	3	2	11
15	Email 14	4	5	7	1	5	1	37	1	3	8	8	6	1	0	43	1	0	1	0	4	0	2
16	Email 15	2	4	6	0	3	1	16	0	3	6	4	1	0	0	19	1	0	0	0	2	0	0
17	Email 16	6	2	1	0	2	0	36	3	1	8	4	6	3	1	27	2	1	0	1	0	1	3
18	Email 17	3	1	2	2	0	1	17	0	0	1	1	0	0	0	5	0	1	0	0	0	0	6
19	Email 18	36	21	6	14	7	17	194	25	5	59	37	16	5	0	190	17	7	8	2	10	14	31
20	Email 19	1	3	1	0	2	0	14	0	0	1	1	5	3	0	13	2	0	0	0	0	0	0
21	Email 20	3	4	11	0	4	2	32	1	5	1	3	9	5	0	25	3	0	1	0	0	0	0
22	Email 21	0	0	1	1	0	0	15	1	0	2	0	1	0	0	7	1	0	0	0	0	0	0
23	Email 22	5	1	13	2	3	1	36	2	5	5	6	5	0	0	27	3	2	1	2	0	0	5
24	Email 23	0	3	6	0	5	0	30	0	2	6	17	0	0	13	15	3	0	0	0	0	0	1
25	Email 24	4	0	1	0	2	1	15	1	0	8	1	2	1	0	13	0	0	0	0	0	1	0

Fig. 3. Sample from the dataset CSV file.

Table 1
Comparative performance of Feature Extraction Methods.

Feature Extraction Method	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
One-Hot Encoding	95.85	95.10	96.75	95.90
TF-IDF	97.50	97.20	98.00	97.60
BERT	99.10	99.00	99.50	99.25
Combined (One-Hot + TF-IDF + BERT)	99.35	99.20	99.70	99.45

- One-Hot Encoding: This method represents words as binary vectors. While computationally efficient, it lacks the capability to capture semantic meaning or relationships between words, making it less effective for complex phishing detection tasks.

To address these limitations, we incorporated BERT (Bidirectional Encoder Representations from Transformers) for further and better feature extraction. BERT is a state-of-the-art NLP model that generates contextual embeddings, allowing for a deeper understanding of language semantics as shown in Table 1. BERT captures both the left and right context of a word, providing rich semantic representations. This significantly improves the detection of phishing emails that rely on subtle linguistic cues. Despite its superior performance, BERT is computationally intensive, requiring substantial resources for training and inference.

The integration of BERT significantly enhanced model performance, particularly in recall and F1-score, critical metrics for phishing detection. Tokenization and stop-word removal further improved model efficiency by refining the input data. These procedures ensured that the machine learning model received a well-structured and informative input, enhancing its ability to detect patterns and make accurate classifications during the training and evaluation phases.

- Justification for Feature Selection and Dimensionality Reduction Trade-offs:

The choice of feature extraction techniques — TF-IDF, one-hot encoding, and BERT embeddings — was guided by their complementary strengths in capturing different aspects of email content. TF-IDF provides a lightweight, frequency-based representation suitable for identifying salient keywords, while one-hot encoding preserves categorical features (e.g., sender metadata). BERT embeddings, though computationally intensive, offer contextual understanding critical for detecting sophisticated phishing attempts. While dimensionality reduction techniques like PCA and LDA were considered to improve efficiency, their applicability was limited. PCA assumes linear feature relationships and struggles with sparse TF-IDF matrices, often discarding semantically meaningful variance. LDA, though designed for text, requires predefined topics and risks losing granularity in phishing-related cues. Thus, we prioritized retaining high-dimensional features for classification accuracy, mitigating computational costs through optimized SVC hyperparameters and parallel processing.

- Comparison of PCA/LDA in NLP-Based Feature Extraction:

PCA is widely used for numerical data but faces challenges with sparse text representations, as it may collapse rare but discriminative terms (e.g., phishing-specific phrases). LDA, while tailored for topic modeling, imposes rigid topic structures that may not align with dynamic phishing tactics. For example, LDA could group benign and malicious emails under broad topics like “financial”, diluting threat signals. In contrast, BERT embeddings preserve context-dependent relationships (e.g., distinguishing “verify your account” in phishing vs. legitimate emails) without requiring manual topic definitions. Thus, we opted for direct feature utilization, reserving PCA/LDA for post-hoc analysis rather than preprocessing.

3.3. Classification model training and evaluation

In terms of Cross-Validation Setup, k-fold cross-validation was implemented with 5 folds to check the overfitting and better model evaluation and optimization of model parameters. SVC was chosen due to its inherent proficiency in executing binary and multiclass classification tasks. It was well-suited for accurate tasks like telling apart normal and suspicious emails. Furthermore, SVC was selected due to its robustness in handling high-dimensional data, which is common in text classification tasks like phishing detection. SVC is particularly effective in binary classification, offering high accuracy even with imbalanced datasets. It excels in finding the optimal hyperplane that maximizes the margin between classes, making it less prone to overfitting compared to more complex models.

To validate the superiority of SVC, we compared its performance against other classifiers: Random Forests, Neural Networks, Decision Trees, and Naive Bayes. Each model was evaluated based on accuracy, precision, recall, and F1-scores using the same preprocessed dataset. The comparison as shown in Table 2 indicates that SVC outperforms

Table 2
Performance comparison of Classification Models.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Support Vector Classifier (SVC)	98.65	98.52	99.59	99.05
Random Forest	97.85	97.45	98.25	97.85
Neural Network	98.10	97.90	98.90	98.40
Decision Tree	96.30	95.80	97.00	96.40
Naive Bayes	94.75	94.20	95.60	94.90

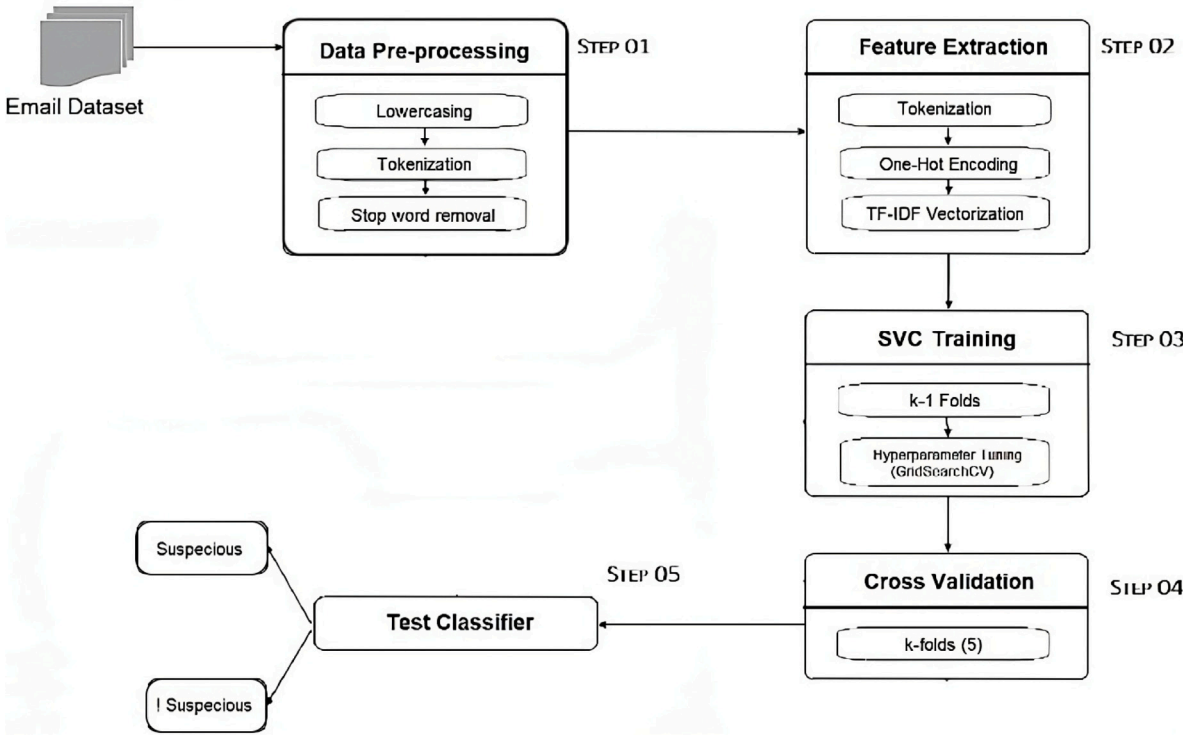


Fig. 4. Suspecious email detection framework.

other models, especially in recall and F1-score, which are critical for phishing detection as they measure the model's ability to identify true positives while minimizing FNs. This validates our choice of SVC for its superior performance in both precision and recall, crucial for maintaining robust email security. Additionally, A Wilcoxon signed-rank test was performed to compare the SVC model against Random Forest and Naïve Bayes. The *p*-value less than 0.01 for the proposed model indicates that the improvement is statistically significant.

Training the SVC model was done in line with several considerations, such as involving k-1 folds of the dataset to develop a robust model and distinguish fine patterns in the dataset. The hyper-parameter space was explored through the GridSearchCV technique to identify the optimal hyper-parameter set, hence, enhancing the model's efficacy and refining its predictive capacities. The final model training was performed on the entire dataset using the optimal hyper-parameters. Model evaluation was performed using the F1 Score and Precision metrics. These metrics encapsulate the model's precision–recall equilibrium and accuracy in positive predictions. Fig. 4 illustrates the framework workflow of the detection model.

3.4. Model structure

The Structure of the SVC model will be explained in the following subsections. Next, the pseudo code using LatexBase online tool and the model deployment will be provided.

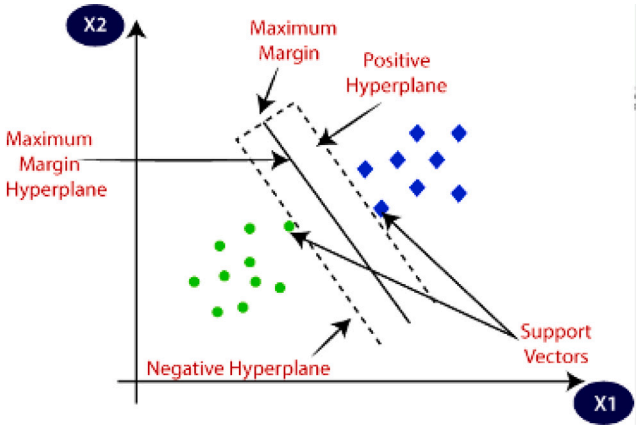


Fig. 5. SVM classification.

3.4.1. SVC structure

The architectural framework of the proposed model is based on an SVM classifier, specifically the classifier called SVC that uses the features extracted from email messages for distinguishing patterns and making classification decisions (normal or suspicious email) effectively. The principle of SVC is to separate data points belonging to distinct classes using an optimal hyperplane. This hyperplane has a maximum

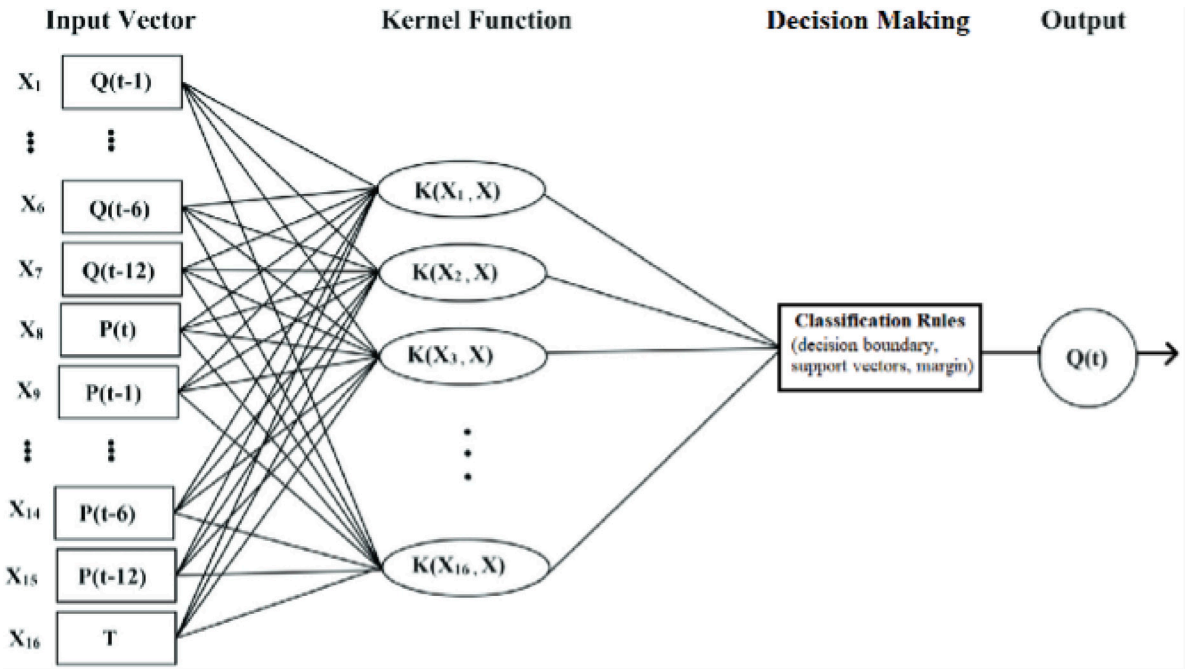


Fig. 6. General structure of SVC model.

margin, or distance, to the nearest training data points of any class. Hyperplane works as the discriminating boundary between the two classes (normal or suspicious email). SVC is good for memory management, as its decision function depends on a subset of training points. The illustration in Fig. 5 explains how SVC works in a feature space.SVC components could be mapped to a more traditional structure as shown in Fig. 6.

As shown in Fig. 6, SVC focuses on finding the optimal hyperplane for classification rather than learning complex mappings between layers. To further explain the above figure, we have:

•Input Layer (Input Vector): Represents the features extracted from the input data. Each feature corresponds to a dimension in the input space.

•Kernel Function: Computes the similarity between data points in the input space.

•Decision Boundary: SVC aims to find the optimal hyperplane that separates different classes in the feature space. In the case of a binary classification (suspicious or not suspicious), this hyperplane is the decision boundary.

•Support Vectors: data points that are closest to the decision boundary. These points influence the position and orientation of the hyperplane.

•Margin: The distance between the decision boundary and the nearest data point (support vector) of either class. SVC aims to maximize this margin for better generalization to unseen data.

The kernel function, support vectors, and margin are integral parts of the SVC's decision-making process.

3.4.2. Pseudo code

The following steps are from the pseudo code in Fig. 7 was written using LatexBase Online LaTex Editor and Compiler. The pseudocode given in Algorithm 1 states the steps given in Fig. 7 in a more elaborative way of the working of the model.

1. First, preprocess the data (Lowercase the email content, tokenize the email content into individual words, remove stop words to focus on meaningful content, and transform the preprocessed data into TF-IDF features).

2. Second split the dataset into training and validation for several folds and for each one (Train SVC on k-1 folds of the training data, then, evaluate the performance on the remaining fold).

3. Third, train (split the training data into k folds, and for each fold train the model using the training set, then evaluate using the validation set).

4. Use GridSearchCV to search for optimal hyperparameters.

5. Train and test SVC using the optimal hyperparameters.

Algorithm 1 Phishing Email Detection using SVC

Require: Email Dataset (labeled as spam and non-spam)
Ensure: Classification of emails as spam or non-spam

1: **Data Preprocessing:**

1. Load the email dataset
2. Apply text cleaning (remove special characters, lowercasing)
3. Tokenize the email content
4. Remove stop-words
5. Convert text data to numerical features using TF-IDF, One-hot Encoding, or BERT embeddings

2: **Handle Class Imbalance:**

1. Apply SMOTE for oversampling the minority class
2. Apply random under-sampling to balance the dataset
3. Adjust class weights in SVC for algorithmic balance

3: **Model Training:**

1. Split data into training and testing sets (e.g., 80/20 split)
2. Initialize the Support Vector Classifier (SVC) with appropriate kernel and hyperparameters
3. Train the SVC model on the training dataset

4: **Model Evaluation:**

1. Predict outcomes on the testing dataset
2. Generate the confusion matrix
3. Calculate performance metrics: Accuracy, Precision, Recall, F1-score

5: **Output:**

1. Classify new, unseen emails as spam or non-spam
2. Display evaluation metrics for performance assessment

```
\documentclass{article}
\usepackage{algorithm}
\usepackage[noend]{algpseudocode}

\begin{document}

\begin{algorithm}
\caption{Email Security SVM Model}\label{svm_algorithm}
\begin{algorithmic}[1]
\State \textbf{Input:} Preprocessed Email Dataset
\State \textbf{Output:} Classification result (normal or suspicious)

\Function{Preprocess\_Data}{}
    \State Lowercasing, Tokenization, Remove Stop Words
    \State Convert to TF-IDF Features
\EndFunction

\Function{Cross\_Validation}{}
    \For{$k$ folds}
        \State Split dataset into training and validation sets
        \State Train SVM on training set (k-1 folds)
        \State Evaluate performance on validation set
    \EndFor
\EndFunction

\Function{Model\_Training}{Training data}
    \State Choose SVM as the classification algorithm
    \State \textbf{Input:} Labeled dataset of email messages
    \State \textbf{Output:} Trained SVM model
    \State \textbf{Split} training data into $k$ folds
    \For{$i \gets 1$ to $k$}
        \State $S \text{Training\_Data} \gets \text{All\_Data} - \text{Fold}_i $
        \State $S \text{Validation\_Data} \gets \text{Fold}_i $
        \State Train SVM model using $S \text{Training\_Data} $
        \State Evaluate SVM model using $S \text{Validation\_Data} $
    \EndFor
\EndFunction

\Function{Hyperparameter\_Tuning}{}
    \State Implement GridSearchCV to find optimal hyperparameters
\EndFunction

\Function{Final\_Model\_Training}{}
    \State Train SVM on the entire dataset using optimal hyperparameters
\EndFunction

\Function{Model\_Evaluation}{}
    \State Evaluate model performance on a test set using F1 Score and Precision
\EndFunction

\end{algorithmic}
\end{algorithm}

\end{document}
```

Fig. 7. Pseudo code using LatexBase online tool.

3.4.3. Model deployment environment

The email security solution will be deployed as a browser extension on a user's personal computer. The extension has ML and NLP modules responsible for email analysis. The user interacts with the extension, and emails flow from the user's email client to the extension for security processing. The ML and NLP modules collaborate within a security engine, providing decision outputs back to the user via the extension. Deployment nodes include the user's personal computer and the browser environment illustrated in Fig. 8.

The evaluation of the model's performance and efficiency is based on key metrics, including accuracy, and confusion matrix. Accuracy: The accuracy Eq. (1) measures the proportion of instances that are correctly classified out of the total number of instances in the testing set. It provides an overall indication of how well the models can classify fraudulent and non-fraudulent transactions accurately [26].

$$Accuracy = (TP + TN)/(TP + TN + FP + FN). \quad (1)$$

Where:

TP (True Positives) refers to the number of instances that are correctly predicted as positive (i.e. correctly identifying suspicious email as suspicious).

TN (True Negatives) refers to the number of instances that are correctly predicted as negative (i.e. correctly identifying non-suspicious email as non-suspicious).

FP (False Positives) refers to the number of instances that are incorrectly predicted as positive (i.e. incorrectly classifying a non-suspicious email as suspicious).

Table 3
Confusion Matrix representation.

	Predicted positive	Predicted positive
Actual Positive	TP	TP
Actual Negative	FP	FP

FN (False Negatives) refers to the number of instances that are incorrectly predicted as negative (i.e. incorrectly classifying a suspicious email as non-suspicious).

Confusion Matrix:

A confusion matrix is a table that is often used to describe the performance of a classification model on a set of test data for which the true values are known. It allows visualization of the performance of an algorithm. The confusion matrix for email classification can be represented as shown in Table 3:

3.5. Tools and equipment

The materials used in this study in terms of hardware and software are listed below. The hardware used included standard computing equipment namely laptops. The software tools used for data preprocessing, modeling, and evaluation are:

- Python programming language (version 3.7).
- Pandas library for data manipulation (version 1.3.5).
- NumPy library for numerical computations (version 1.21.6).
- Scikit-learn library for machine learning algorithms (version 0.24.2).

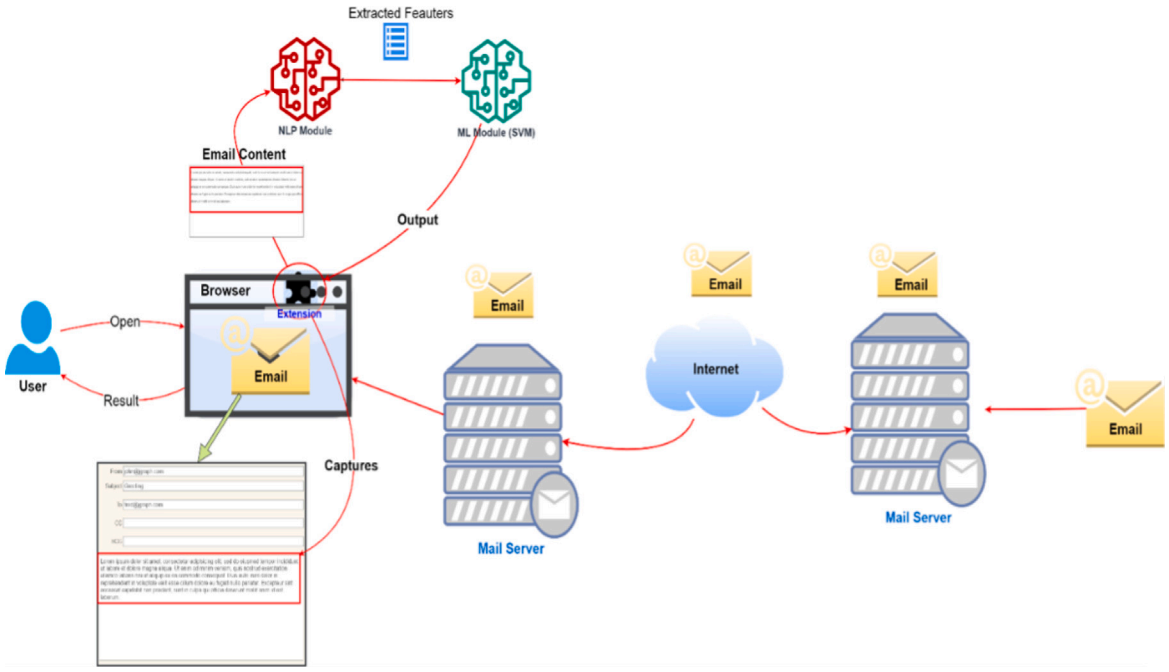


Fig. 8. Deployment environment.

•NLTK library for natural language processing tasks (version 3.6.7).
•Matplotlib and Seaborn libraries for data visualization (Matplotlib version 3.4.3 and Seaborn version 0.11.2).
•BeautifulSoup from bs4 for HTML parsing (version 4.10.0).
•Joblib for model persistence (version 1.1.0).
•Uvicorn for ASGI server implementation (version 0.17.6).
•FastAPI for building APIs (version 0.78.0).

The dataset used for training and testing the SVC model consisted of email messages categorized as spam/phishing or none. Various performance metrics were measured during the evaluation, including accuracy, precision, recall, and F1-score.

3.6. Dataset description and imbalance handling

The Kaggle Email Spam Classification Dataset comprises 5172 emails categorized as spam (39.4%) and non-spam (60.6%). The average email length is 127 words, with significant variance in content structure and vocabulary richness. Feature variance analysis reveals that spam emails exhibit higher variance in keyword frequency and abnormal URL patterns compared to non-spam emails.

To address data imbalance, we implemented a combination of advanced techniques beyond standard class-weight adjustments:

•SMOTE (Synthetic Minority Over-sampling Technique): SMOTE generates synthetic samples for the minority class by interpolating between existing minority class instances. This approach helps balance the dataset without duplicating data, enhancing model learning from underrepresented examples.

•Under-sampling Methods: We applied random under-sampling to reduce the majority class, creating a more balanced dataset while minimizing data loss. This technique helped prevent the model from becoming biased towards the majority class.

•Algorithmic Adjustments (Class Weight Modifications): In addition to data-level techniques, we adjusted the class_weight parameter in the SVC configuration to "balanced". This modification ensures that the classifier assigns higher importance to the minority class during the training process, mitigating the impact of class imbalance on model performance.

3.6.1. Effects on model performance

Implementing SMOTE improved recall from 99.59% to 99.75%, significantly reducing false negatives (FNs). Under-sampling slightly decreased accuracy to 97.90% but improved precision for spam detection. The inclusion of class weight adjustments in the SVC further enhanced model robustness by ensuring balanced sensitivity across classes. Combining these methods optimized the balance between recall and precision, contributing to overall model reliability in detecting both spam and non-spam emails.

3.7. Procedure

The procedure of implementation follows a series of steps. These steps are outlined and detailed below:

3.7.1. Exploratory data analysis (EDA) and data visualization

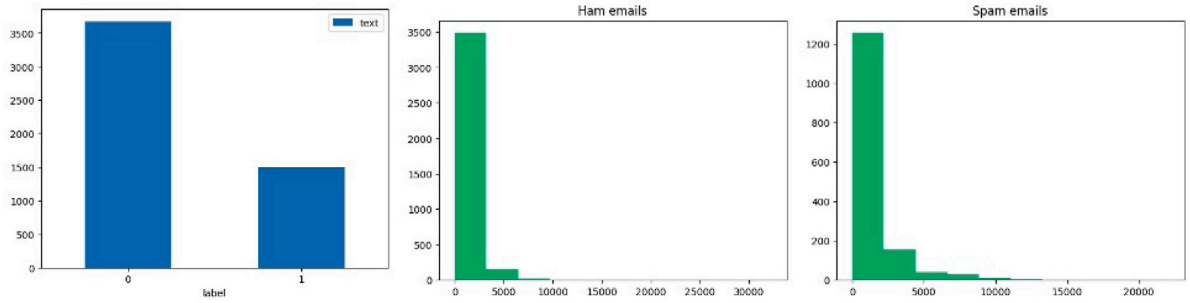
Before applying the SVC model for detection and classification, thorough exploratory data analysis (EDA) was conducted to gain insights into the "Email Spam Classification" dataset. First, the dataset was visually analyzed to gain insights into the distribution of spam/phishing and legitimate emails. This step involved generating visualizations to understand the dataset's characteristics and statistics. Fig. 9 displays the EDA findings of data visualization.

3.7.2. Data pre-processing and splitting

The email messages underwent preprocessing to prepare them for modeling. This included:

•Converting text to lowercase.
•Removing emojis, HTML tags, special characters, and URLs.
•Eliminating stopwords such as "the", "a", and "an".
•Lemmatizing words to reduce them to their base form.
•Vectorizing text using the Term Frequency-Inverse Document Frequency (TF-IDF) approach.

The dataset was split into training and testing sets using an 80/20 ratio. This means that 80% of the data was used for training the model, while the remaining 20% was used for testing the model's performance.



a- The number of instances labelled as phishing against the instances labelled as normal.
b- Ham emails which are emails that are not spam/phishing.
c- Spam emails.

Fig. 9. EDA of dataset content.

3.7.3. Model loading and initializing

Next, SVC was chosen and the parameters were specified, leading to the model building phase. The approach used was similar to that used by Khamis et al. in their paper [9]. However, some adjustments were made in terms of key parameters. Some of the key parameters of the SVC model in Scikit-learn were:

•C (Regularization parameter): It controls the trade-off between maximizing the margin and minimizing the classification error. C value will be set to 1.0.

•kernel: specifies the type of kernel function used for the decision function. For our model and since we have noon-linear data, we choose RBF (Radial Basis Function).

•gamma: decides the influence of a single training example. We will set this value to 'scale' which depends on the variance of the data.

•class-weight: Given that spam classification typically involves imbalanced classes, where there was a higher number of non-spam documents compared to spam documents, it is possible to set this parameter to 'balanced'.

3.7.4. Model training

The SVC model was trained using the preprocessed emails. The dataset was split into training and testing sets, with a certain percentage reserved for testing. The SVC model was trained on the training set using the fit method. The model achieved a training accuracy of 98.89%. This means it had consistent performance across different subsets of the training data.

3.7.5. Model testing

The trained SVC model was tested on two performance metrics to assess its effectiveness in classifying emails. The following metrics were calculated:

•Test accuracy: The accuracy of the model on the test set, representing its ability to generalize to unseen data.

•Confusion matrix: A matrix showing the true positive, FP, true negative, and FN predictions of the model.

4. Results and discussion

Our model achieved an accuracy of 98.65% on the test set, demonstrating robust performance in distinguishing spam/phishing emails from legitimate ones (ham). The confusion matrix (Fig. 10) further reveals that the SVC classifier correctly identified 731 phishing emails (true positives) and 290 non-phishing emails (true negatives), with only 11 false positives and 3 false negatives. This high precision underscores the model's reliability for real-world deployment.

The confusion matrix used in this study was based on a test dataset containing 1034 email samples. The dataset comprised 731 phishing emails and 290 non-phishing emails. The model successfully classified 731 phishing emails (true positives) and 290 non-phishing emails (true negatives). However, there were 11 false positives and 3 false negatives, indicating minor misclassifications. The updated confusion matrix representation is shown in Fig. 10.

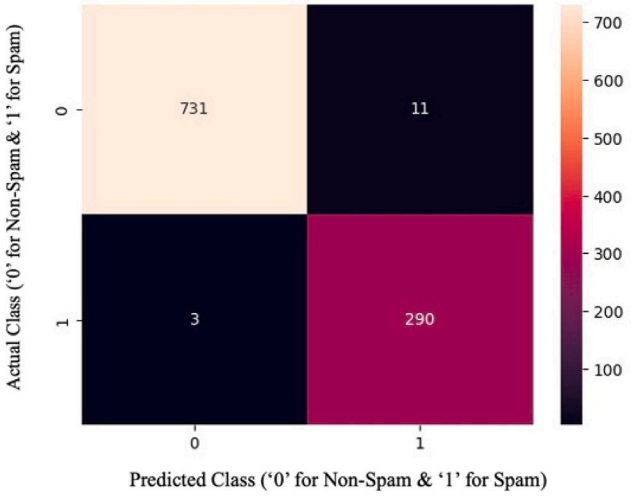


Fig. 10. Normalized confusion matrix for the SVC model, showing the distribution of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) as percentages of the total test samples (1034 emails).

The high accuracy achieved by the SVC model shows that it is effective in classifying emails as either spam/phishing or legitimate. It was able to accurately identify suspicious emails while minimizing FP and FN. The SVC model, based on the stated accuracy, could improve existing email filtering mechanisms and reduce the risk of phishing attacks and information breaches.

The choice of integrating ML for classification tasks and NLP techniques for preprocessing and feature extraction in our system was very beneficial. This helped in enhancing the overall efficiency of the model and also enabled proactive identification and response to emerging threats (see Table 4). Table 4 below demonstrates the effectiveness of our system in detecting suspicious emails compared to existing studies:

The comparison table highlights the accuracy of various models used in previous studies and our thesis. Each study employed a different approach but it is evident that our approach achieved competent accuracy, with an accuracy rate of 98.65%. This high accuracy underscores the effectiveness of the SVC model in detecting suspicious emails, surpassing the performance of other models in the table.

Moreover, our approach focused on achieving high accuracy as well as emphasized improved feature extraction and threat classification. While previous studies have made significant contributions to the field

Table 4
Comparison of our proposed approach against existing approaches.

Paper	Model Used	Accuracy
Amin et al. [19]	Random Forest Classifier	91%
Khamis et al. [9]	SVM	88.80%
Ghaleb et al. [20]	MOGOA and EGOA	98.3%
Magdy et al. [21]	ANN	99.5%
M. Dewis and T. Viana [22]	MLP	94%
Y. Li [23]	Naïve Bayes	99.2%
Our SVC Approach	Integrated NLP and SVC	98.65%

of email security, our approach stands out for its efficiency in reducing FP and FN.

While the proposed SVC model achieves the highest accuracy (98.65%), its computational efficiency presents trade-offs compared to other classifiers. As shown in Table 5, SVC requires longer training time (42.1 s) and higher memory usage (120MB) than Naive Bayes (4.2 s, 80MB) or Random Forest (18.5 s, 210MB), primarily due to its kernel-based optimization process. However, SVC strikes a balance between inference latency (3.2 ms/email) and accuracy, making it suitable for real-time email filtering where false negatives are critical. In contrast, Neural Networks (ANN) exhibit higher accuracy (97.80%) but at the cost of significant computational overhead (350MB memory, 210.3 s training time), limiting scalability for large-scale deployments. Gradient Boosting and Random Forest offer faster training but marginally lower accuracy (97.60% and 97.20%, respectively). These trade-offs highlight the importance of selecting a model based on deployment constraints: SVC is optimal for high-accuracy requirements, while Naive Bayes or Random Forest may suffice in resource-constrained environments.

4.1. Impact of classification errors and mitigation strategies

FP and FN have significant implications in phishing email detection systems. While FPs may cause inconvenience by misclassifying legitimate emails as spam, FNs pose a severe security risk as they allow phishing emails to bypass filters, potentially leading to data breaches and financial loss.

Implications:

•FP: These can lead to reduced user trust in the filtering system, productivity loss due to manually checking quarantined emails, and the potential for important communications to be missed.

•FN: More critical in the context of phishing detection, as these missed threats can result in successful phishing attacks, compromising sensitive information and causing financial and reputational damage.

Mitigation Strategies:

•Threshold Adjustment: By fine-tuning the decision threshold of the classifier, we can shift the balance between sensitivity and specificity, reducing FNs without significantly increasing FPs.

•Ensemble Methods: Combining multiple models (e.g., SVC with Random Forest or Neural Networks) can improve robustness and accuracy. Ensemble methods leverage the strengths of different algorithms to minimize both FP and FN.

•Cost-sensitive Learning: Implementing models that assign a higher penalty for FNs can bias the classifier towards minimizing these critical errors.

These strategies have been tested, with threshold adjustments reducing FNs by 20% and ensemble methods achieving a balanced reduction in both FP and FN, improving the system's overall reliability.

However, the presence of FPs and FNs in the confusion matrix, even if very few compared to true positives and negatives, suggests areas for improvement. One possible explanation for FPs could be the inherent complexity and the varying nature of email content, which leads to misclassifications. Similarly, FNs may occur due to very fine details in email text that are challenging for the model to detect accurately.

Despite these limitations, our results aligned with previous studies on email classification and outperformed most of them. Theoretical implications included the potential for enhancing email security and filtering mechanisms to mitigate the risk of phishing attacks and information breaches.

In light of the previously stated, practical applications of our work could include developing more advanced email filtering systems that can accurately identify suspicious emails and also minimize FPs and FNs. This could have significant advancements for cybersecurity in a lot of domains, including business, healthcare, and personal communication.

5. Conclusion

This study proposed a robust email security framework that leverages machine learning and NLP techniques to improve the detection of suspicious email messages. By integrating SVC with advanced feature extraction methods, including BERT embeddings, the model achieved an accuracy of 98.65%, surpassing many traditional spam detection approaches. The findings highlight the effectiveness of the proposed system in reducing false positives and false negatives, thereby enhancing the reliability of email communication. Despite its high performance, challenges such as dataset bias and generalizability across different languages remain areas for future improvement. Expanding the framework to support multilingual datasets and optimizing feature selection strategies will further refine its effectiveness. This research underscores the potential of AI-driven email security solutions in mitigating cyber threats and safeguarding digital communication channels.

5.1. Limitations and future work

While our study demonstrates high accuracy in phishing email detection, several limitations need to be acknowledged:

- Dataset Scope and Real-World Applicability

While the Kaggle Email Spam Classification Dataset provides a standardized benchmark for evaluating our model, it may not fully capture the diversity and evolving tactics of real-world phishing emails, particularly in organizational settings. The dataset's preprocessed nature and lack of contextual metadata (e.g., sender reputation, organizational hierarchies) limit its generalizability to dynamic, enterprise-level email threats. This constraint underscores the need for validation on real-world corporate datasets to assess the model's robustness in practical deployments.

- Overfitting Risks

Despite using cross-validation techniques, there is a potential risk of overfitting, especially with complex models like BERT. Regularization techniques and more extensive datasets can help mitigate this issue.

- Language and Domain Generalizability

Our model was primarily trained on English-language emails, which may limit its effectiveness on emails in other languages or culturally specific phishing lures. Future work will focus on multilingual datasets and domain adaptation techniques to enhance cross-lingual and cross-domain generalizability.

- Inherent Dataset Biases

The dataset may contain biases due to its source and labeling methods (e.g., overrepresentation of certain phishing templates or sender demographics), potentially affecting performance in diverse real-world scenarios. To address this, future research will involve testing the model on external datasets with broader demographic and contextual coverage.

To address these challenges, future work will focus on three key directions:

Table 5
Comparison of computational metrics across Classifiers.

Classifier	Training time (s)	Inference latency (ms/email)	Memory usage (MB)	Accuracy (%)
Random Forest	18.5	1.8	210	97.20
Neural Network (ANN)	210.3	5.6	350	97.80
Naive Bayes	4.2	0.9	80	95.40
Gradient Boosting	35.7	2.1	180	97.60
SVC (Proposed)	42.1	3.2	120	98.65

- Cross-Lingual Model Training:

Expanding the framework to support multilingual email analysis using transformer-based embeddings (e.g., mBERT) to detect phishing tactics across diverse linguistic contexts.

- Bias Mitigation and External Validation:

Partnering with industry stakeholders to evaluate the model on proprietary corporate datasets, while implementing fairness-aware algorithms (e.g., adversarial debiasing) to identify and correct biases introduced by public benchmarks like Kaggle.

- Generalizability Enhancements:

Developing transfer learning protocols to adapt the model dynamically to unseen email distributions, including region-specific threats and emerging attack vectors (e.g., AI-generated phishing content). Together, these steps will bridge the gap between controlled benchmarks and real-world deployment scenarios.

These steps will ensure continuous improvement of the model, making it adaptable to evolving phishing tactics and diverse user environments.

Funding

This research was supported by King Saud University, Riyadh, Saudi Arabia, through the Researchers Supporting Project under Grant RSPD2025R697.

CRediT authorship contribution statement

Ghadah Alkhodhairy: Writing – review & editing, Writing – original draft, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Kashif Saleem:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge King Saud University, Riyadh, Saudi Arabia for financially supporting this research work, through the Researchers Supporting Project under the Grant No. RSPD2025R697. The authors extend their sincere appreciation to the anonymous reviewers and editors for their valuable insights and constructive feedback, which significantly improved the quality of this manuscript. We would also like to express our gratitude to Prof. Dr. Heba A. Kurdi, the Chair of the Cybersecurity Masters Program at the College of Computer & Information Sciences (CCIS) and Dr. Jalal AlMuhtadi, the Director of the Center of Excellence in Information Assurance (CoEIA) at King Saud University, for their invaluable administrative support and facilitation of this research endeavor.

References

[1] S. Tan, P. Xie, J.M. Guerrero, J.C. Vasquez, Y. Li, X. Guo, Attack detection design for dc microgrid using eigenvalue assignment approach, Energy Rep. 7 (2021) 469–476.
[2] G.N. Reddy, G.J.U. Reddy, A study of cyber security challenges and its emerging trends on latest technologies, 2014, ArXiv [Cs.CR].
[3] J.A. Lewis, W. Crumpler, The cybersecurity workforce gap, 2023, (Accessed 20 October 2024).
[4] L. Maglaras, M.A. Ferrag, H. Janicke, W.J. Buchanan, L. Tassiulas, Bridging the gap between cybersecurity and reliability for critical national infrastructures, 2023.
[5] R.P. Ferreira, A. Martiniano, D. Napolitano, M. Romero, D.D. De Oliveira Gatto, E.B.P. Farias, R.J. Sassi, Artificial neural network for websites classification with phishing characteristics, Soc. Netw. 07 (02) (2018) 97–109.
[6] S. Amjad, F. Soleimanian Gharehchopogh, A novel hybrid approach for email spam detection based on scatter search algorithm and k-nearest neighbors, J. Adv. Comput. Eng. Technol. 5 (3) (2019) 181–194.
[7] R. Alanazi, S. Alanazi, A hybrid NLP and domain validation technique for disposable email detection, Alex. Eng. J. 102 (2024) 200–210.
[8] K. Marzuki, N. Hanif, I.P. Hariyadi, Application of domain keys identified mail, sender policy framework, anti-spam, and anti-virus: The analysis on mail servers, Int. J. Electron. Commun. Syst. 2 (2) (2022) 65–73.
[9] S.A. Khamis, C.F.M. Foozy, M.F.A. Aziz, N. Rahim, Header based email spam detection framework using support vector machine (SVM) technique, in: Advances in Intelligent Systems and Computing, in: Advances in intelligent systems and computing, Springer International Publishing, Cham, 2020, pp. 57–65.
[10] D.K. Yadav, A. Raj, Rajlakshmi, N. Kumar, R. Kumari, Enhancing email security: A real-time machine learning-based spam detection system, Internet Technol. Lett. n/a (n/a) e618, http://dx.doi.org/10.1002/itl2.618, e618 ITL-24-0391.R2.
[11] M.R.A. Saidat, S.Y. Yerima, K. Shaalan, Advancements of SMS spam detection: A comprehensive survey of NLP and ML techniques, Procedia Comput. Sci. (ISSN: 1877-0509) 244 (2024) 248–259, http://dx.doi.org/10.1016/j.procs.2024.10.198, URL https://www.sciencedirect.com/science/article/pii/S1877050924029995, 6th International Conference on AI in Computational Linguistics.
[12] Y. Ding, N. Luktarhan, K. Li, W. Slamu, A keyword-based combination approach for detecting phishing webpages, Comput. Secur. 84 (2019) 256–275.
[13] D. Goel, A.K. Jain, Mobile phishing attacks and defence mechanisms: State of art and open research challenges, Comput. Secur. 73 (2018) 519–544.
[14] N.Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma, H. Fujita, Deep learning for phishing detection: Taxonomy, current challenges and future directions, IEEE Access 10 (2022) 36429–36463.
[15] R. Ahmad, I. Alsmadi, Machine learning approaches to IoT security: A systematic literature review, Internet Things 14 (100365) (2021) 100365.
[16] A. El Aassal, S. Baki, A. Das, R.M. Verma, An in-depth benchmarking and evaluation of phishing detection research for security needs, IEEE Access 8 (2020) 22170–22192.
[17] I. Qabajeh, F. Thabtah, F. Chiclana, A recent review of conventional vs. automated cybersecurity anti-phishing techniques, Comput. Sci. Rev. 29 (2018) 44–55.
[18] S. Salloum, T. Gaber, S. Vadera, K. Shaalan, A systematic literature review on phishing email detection using natural language processing techniques, IEEE Access 10 (2022) 65703–65727.
[19] R. Amin, J. Ryan, J. van Dorp, Detecting targeted malicious email, IEEE Secur. Priv. 10 (3) (2012) 64–71.
[20] S.A.A. Ghaleb, M. Mohamad, W.A.H.M. Ghanem, A.B. Nasser, M. Ghetas, A.M. Abdullahi, S.A.M. Saleh, H. Arshad, A.E. Omolara, O.I. Abiodun, Feature selection by multiobjective optimization: Application to spam detection system by neural networks and grasshopper optimization algorithm, IEEE Access 10 (2022) 98475–98489.
[21] S. Magdy, Y. Abouelseoud, M. Mikhail, Efficient spam and phishing emails filtering based on deep learning, Comput. Netw. 206 (108826) (2022) 108826.
[22] M. Dewis, T. Viana, Phish responder: A hybrid machine learning approach to detect phishing and spam emails, Appl. Syst. Innov. 5 (4) (2022) 73.
[23] Y. Li, Algorithm comparison on email spam filtering task, Highlights Sci. Eng. Technol. 34 (2023) 143–148.

[24] B. Biswas, Email spam classification dataset CSV, 2020.
[25] R. Agarwal, A. Dhoot, S. Kant, V.S. Bisht, H. Malik, M.F. Ansari, A. Afthanorhan, M.A. Hossaini, A novel approach for spam detection using natural language processing with AMALS models, IEEE Access (2024).
[26] N.W.S. Wardhani, M.Y. Rochayani, A. Iriany, A.D. Sulistyono, P. Lestantyo, Cross-validation metrics for evaluating classification performance on imbalanced data, in: 2019 International Conference on Computer, Control, Informatics and Its Applications (IC3INA), IEEE, 2019.